Labo 12 — Pods, Deployments et probes

Théorie — comment on fait tourner une application dans un cluster sans jamais créer un Pod à la main : des réplicas, des étiquettes, une mise à jour qui ne coupe rien, un retour arrière en une commande, et des sondes qui disent au cluster ce qu'est « prêt ».

Objectifs

- Comprendre la chaîne **Deployment → ReplicaSet → Pod** et le rôle des labels et selectors.
- Mettre à l'échelle, mettre à jour (*rolling update*) et revenir en arrière (`rollout undo`).
- Distinguer les trois sondes — readiness, liveness, startup — et leurs effets.
- Lire un rollout bloqué et savoir pourquoi les anciens Pods continuent de servir.
- Relier tout cela au `HEALTHCHECK` et aux `--restart` des labos Docker.

---

1. Personne ne crée un Pod à la main

Au labo 11, un Pod effacé ne revenait pas, mais un Pod sous Deployment réapparaissait. La raison tient en trois objets emboîtés :

Objet	Ce qu'il décrit	Qui le crée
Pod	un ou plusieurs conteneurs qui tournent ensemble	le ReplicaSet
ReplicaSet	« N copies de ce modèle de Pod doivent exister »	le Deployment
Deployment	« voici le modèle de Pod, le nombre de réplicas, et comment passer d'une version à l'autre »	**vous**

Le Deployment porte un **template** de Pod (`spec.template`) et un nombre de réplicas. À chaque changement du template (nouvelle image, nouvelle variable), il crée un **nouveau** ReplicaSet et transfère les réplicas de l'ancien vers le nouveau, progressivement. Les anciens ReplicaSets restent à `0` réplica : c'est l'historique qui permet le retour arrière. Vous ne manipulez jamais un ReplicaSet directement ; vous ne créez un Pod seul que pour un test.

> **→ RAPPEL DOCKER**
> Un Deployment, c'est `podman run` plus `--restart=always` plus `--scale` plus un script de mise à jour sans coupure, écrits une fois pour toutes dans un objet que le cluster exécute. Le template de Pod contient ce que vous mettiez dans la ligne `podman run` : image, ports, variables, volumes. La politique `restartPolicy: Always` du template reprend le `--restart` du labo 03 — appliquée par le kubelet, qui lui ne disparaît jamais.

2. Labels et selectors : comment les objets se trouvent

Un ReplicaSet ne « contient » pas ses Pods : il les **sélectionne** par leurs labels. Le Deployment déclare `selector.matchLabels: {app: api}` et son template porte `labels: {app: api}` ; tout Pod portant ce label et appartenant au ReplicaSet compte comme un réplica. Le cluster ajoute un label `pod-template-hash` à chaque Pod, calculé depuis le template : c'est ce qui distingue les Pods de la version 1.0 de ceux de la version 1.1, et ce qui donne aux noms leur forme `api-6c7bb54fc5-2th65` (nom du Deployment, hash du template, suffixe aléatoire).

Les labels sont le mécanisme universel de Kubernetes : les Services (labo 13) choisissent leurs Pods de la même façon, et `kubectl get pods -l app=api` filtre comme le ferait un contrôleur. Un label est une paire clé/valeur libre ; les conventions d'entreprise en fixent quelques-unes (`app`, `version`, `tier`, `environment`), souvent sous le préfixe `app.kubernetes.io/`.

3. Mettre à l'échelle et mettre à jour

`kubectl scale deployment api --replicas=3` change un chiffre dans la spec ; le ReplicaSet crée ou supprime des Pods pour l'atteindre. En production, ce chiffre est fixé dans le manifeste (et modifié par `apply`), ou piloté par un HorizontalPodAutoscaler.

La mise à jour est l'opération la plus importante. `kubectl set image deployment/api api=localhost/api:1.1` (ou un `apply` du manifeste modifié) déclenche un **rolling update** : le Deployment crée un ReplicaSet pour la nouvelle version, y ajoute un Pod, attend qu'il soit **prêt**, retire un ancien Pod, et recommence. Deux réglages de `strategy.rollingUpdate` bornent le mouvement : `maxSurge` (combien de Pods en plus du nombre voulu, 25 % par défaut) et `maxUnavailable` (combien peuvent manquer, 25 % par défaut). Avec trois réplicas, le cluster ne descend jamais sous deux Pods prêts, et le service ne coupe pas.

`kubectl rollout status` suit l'opération, `rollout history` liste les révisions, `rollout undo` revient à la précédente (ou `--to-revision=N`). Chaque révision est un ReplicaSet conservé à zéro ; `revisionHistoryLimit` (10 par défaut) borne leur nombre.

> **À RETENIR**
> Un rollout qui échoue n'est pas une panne : si les nouveaux Pods ne deviennent jamais prêts, le Deployment **s'arrête** après le premier (ou les `maxSurge` premiers) et laisse les anciens servir. `kubectl rollout status` expire, `kubectl get deployment` montre `UP-TO-DATE 1 / AVAILABLE 3`, et `rollout undo` ramène tout en quelques secondes. C'est ce que `compose up` ne sait pas faire : il remplace, et vous découvrez la panne après.

4. Les sondes : dire au cluster ce qu'est « prêt » et « vivant »

Le rolling update repose sur une notion : un Pod **prêt**. Sans sonde, le cluster considère un conteneur prêt dès que son processus est démarré — pour une JVM, dix secondes avant qu'elle n'accepte une requête. Les **probes** corrigent cela. Trois existent, avec trois effets différents :

Sonde	Question posée	Si elle échoue	Usage
readinessProbe	« peux-tu recevoir du trafic ? »	le Pod est retiré des Services, **pas redémarré** ; `READY 0/1`	démarrage lent, dépendance temporairement absente
livenessProbe	« es-tu encore vivant ? »	le conteneur est **tué et redémarré** (`RESTARTS` augmente)	blocage, fuite mémoire, boucle infinie
startupProbe	« as-tu fini de démarrer ? »	les deux autres sont suspendues jusqu'au succès	applications très lentes au démarrage

Chaque sonde est une commande (`exec`), une requête HTTP (`httpGet`) ou une connexion TCP (`tcpSocket`), avec `initialDelaySeconds`, `periodSeconds`, `failureThreshold`. Elles sont exécutées par le **kubelet**, depuis le nœud, sans rien installer dans l'image — c'est l'avantage sur le `HEALTHCHECK` du labo 10, qui exigeait `curl` ou `wget` dans le conteneur.

→ SPRING BOOT
Spring Boot expose deux endpoints faits pour cela : `/actuator/health/readiness` (l'application est-elle prête à servir : pool de connexions ouvert, caches chargés) et `/actuator/health/liveness` (l'application est-elle dans un état sain). Activés par `management.endpoint.health.probes.enabled=true` (automatique quand Spring détecte Kubernetes), ils reflètent l'état interne (`ApplicationAvailability`) et non un simple « le port répond ». Notre fausse API n'a que `/actuator/health` ; les sondes de ce labo l'utilisent pour les deux.

PIÈGE
Une **liveness** mal réglée est plus dangereuse qu'aucune sonde. Si elle teste une dépendance (la base de données), une panne de la base fait redémarrer l'API en boucle, ce qui n'aide en rien. Si son délai est trop court, une JVM lente est tuée avant d'avoir démarré, puis à nouveau, puis `CrashLoopBackOff`. Et surtout : une liveness qui échoue sur la **nouvelle** version ne bloque pas le rollout — les nouveaux Pods passent `Ready` avant que la liveness ne les tue, les anciens sont retirés, et **tout le service** tombe. La readiness protège le déploiement ; la liveness protège contre les blocages. Ne confondez pas leurs rôles.

5. Lire l'état d'un Pod

`kubectl get pods` résume tout en trois colonnes : `READY` (conteneurs prêts / total), `STATUS`, `RESTARTS`. Les états à reconnaître :

`STATUS`	Signification	Premier réflexe
`Pending`	pas encore placé ou en attente d'image/volume	`describe` → Events
`ContainerCreating`	image en cours de téléchargement, volumes en montage	attendre, puis `describe`
`Running` `0/1`	tourne mais pas prêt : readiness en échec	`describe` → `Readiness probe failed`
`Error` / `CrashLoopBackOff`	le processus sort en erreur, relancé avec un délai croissant	`kubectl logs --previous`
`ImagePullBackOff`	image introuvable (labo 11)	nom, tag, import
`Terminating`	en cours d'arrêt (`SIGTERM`, puis `SIGKILL` après `terminationGracePeriodSeconds`, 30 s)	normal pendant un rollout

`CrashLoopBackOff` mérite une précision : ce n'est pas un état de l'application mais du **kubelet**, qui attend de plus en plus longtemps (10 s, 20 s, 40 s… jusqu'à 5 min) avant de relancer un conteneur qui ne cesse de sortir. `kubectl logs pod --previous` montre la sortie de la tentative précédente : c'est là qu'est la cause.

K3S
Rien de propre à k3s dans ce labo : Deployments, ReplicaSets et sondes sont le cœur de l'API, identiques partout. La seule différence visible est la **vitesse** : sur un nœud unique, les images sont déjà là et un rollout de trois réplicas prend dix secondes ; sur un cluster managé, chaque nouveau Pod peut attendre un nœud, un pull depuis le registry et un équilibreur, et le même rollout prend quelques minutes.

6. En entreprise

- **Tout service est un Deployment** (ou un StatefulSet pour les bases, labo 15). Le manifeste est versionné, et une mise en production est un `apply` avec une nouvelle image — par digest, labo 10.
- **Deux réplicas minimum** en production, pour qu'un rollout ou une panne de nœud ne coupe rien ; trois pour les services critiques.
- **Readiness obligatoire, liveness réfléchie** : la readiness sur `/actuator/health/readiness`, la liveness sur `/actuator/health/liveness` avec un délai généreux, jamais sur une dépendance.
- **Le rollback est un réflexe, pas un drame** : `rollout undo` fait partie de la procédure d'incident, et `rollout history` de la revue post-incident.
- **Les ressources** (`requests` / `limits`, labo 16) complètent le template : sans elles, le scheduler place à l'aveugle.

À retenir

- Deployment → ReplicaSet → Pod : vous écrivez le premier, le cluster crée les deux autres ; les labels les relient.
- `scale` change un chiffre ; `set image` ou `apply` d'un nouveau template déclenche un rolling update borné par `maxSurge` et `maxUnavailable`.
- Un rollout dont les Pods ne deviennent pas prêts s'arrête et laisse les anciens servir ; `rollout undo` revient en arrière.
- Readiness = retirer du trafic sans redémarrer ; liveness = redémarrer ; startup = attendre avant de juger.
- Une liveness mal réglée tue un service sain ; une readiness en échec sur la nouvelle version bloque le rollout sans dommage.
- `CrashLoopBackOff` est un délai du kubelet ; la cause est dans `kubectl logs --previous`.
- Les sondes sont exécutées par le kubelet, sans outil dans l'image — contrairement au `HEALTHCHECK` Docker.

Vocabulaire

Deployment : objet qui décrit un template de Pod, un nombre de réplicas et une stratégie de mise à jour. — **ReplicaSet** : contrôleur qui maintient N Pods identiques ; créé par le Deployment. — **template** : le modèle de Pod dans `spec.template`. — **label / selector** : étiquette clé/valeur / filtre sur les labels. — **`pod-template-hash`** : label calculé qui identifie une version du template. — **rolling update** : remplacement progressif des Pods ; **`maxSurge`** / **`maxUnavailable`** : ses bornes. — **révision** : un ReplicaSet conservé pour `rollout undo`. — **readiness / liveness / startup probe** : sondes du kubelet et leurs effets. — **`CrashLoopBackOff`** : délai croissant avant la relance d'un conteneur qui sort en erreur. — **`terminationGracePeriodSeconds`** : délai entre `SIGTERM` et `SIGKILL`.